

Feasibility studies

A collection of possible features for TiefDownConverter

Planning Documents

Below the planned feature feasibility study, including use cases and usefulness.

Linear Chaining of Preprocessors — Feasibility Study

Abstract

TiefDownConverter’s current preprocessor pipeline selects exactly one preprocessor per file-extension group and executes it as a single external process. This study assesses the feasibility of extending that pipeline to support *linear chaining*: executing an ordered sequence of preprocessors per file or file group, where the standard output of each step is fed as input to the next. The study covers architectural impact, manifest model changes, implementation complexity, testability, and practical use cases.

1. Background and Motivation

A PreProcessor in TiefDownConverter is a record of a CLI command (`cli`, defaulting to `pandoc`) and its arguments (`cli_args`). At conversion time, `choose_preprocessor` selects one preprocessor whose `extension_filter` matches the current file-extension chunk, and `run_preprocessor` invokes it, capturing its standard output as the chunk’s result string.

This design is sufficient for single-step transformations (e.g., converting Markdown to LaTeX via `pandoc`). It becomes limiting when a workflow requires composing independent transformation steps — for example, normalising a source format before the primary conversion, or post-processing `pandoc`’s output with a lightweight tool before writing it to disk. Currently, such compositions must be encoded as a single monolithic script, coupling unrelated concerns and reducing reusability.

2. Current Architecture

The relevant call chain is:

```
run_preprocessors_on_inputs
├─ get_preprocessing_chunks          // groups files by extension
│   └─ [par_iter] choose_preprocessor // selects ONE PreProcessor per chunk
│       └─ run_preprocessor          // spawns process, captures stdout
```

`run_preprocessor` uses `Stdio::piped()` for stdout capture and passes input file paths as positional CLI arguments. There is no mechanism to route the captured output back into another process.

3. Proposed Extension

3.1 Conceptual Model

A *preprocessor chain* is an ordered, non-empty list of preprocessor names. Executing a chain over a set of input files means:

1. Run the first preprocessor with the original input files as arguments; capture its stdout as an intermediate string.
2. For each subsequent preprocessor, write the intermediate string to a named temporary file inside the compiled directory, pass that file as the sole argument, capture the new stdout.
3. Return the final stdout string as the chunk result.

The single-preprocessor case remains unchanged: a chain of length one degenerates to the existing behaviour.

3.2 Manifest Model Changes

A new struct `PreProcessorChain` is introduced:

```
[[custom_processors.preprocessor_chains]]
name           = "md_to_html_clean"
extension_filter = "md"           # optional, same semantics as PreProcessor
steps          = ["pandoc_html", "html_tidy"]
```

`Processors` gains a `preprocessor_chains: Vec<PreProcessorChain>` field. A template's `PreProcessors.preprocessors` list references chain names exclusively — chains become the sole dispatch unit. Individual `PreProcessor` entries remain in `Processors.preprocessors` as building-block definitions but are no longer referenced directly by templates.

3.3 Execution Changes

`run_preprocessor` is split into two functions:

- `run_preprocessor_step(step, compiled_dir, args, stdin_source)` — runs one process, either with file paths or with a temp-file path derived from `stdin_source`.
- `run_preprocessor_chain(chain, compiled_dir, metadata, input_files)` — iterates over the steps, threading the intermediate output through.

Temporary files for intermediate results are placed in the compiled directory with a deterministic name (e.g., `.chain_intermediate_{chunk_hash}.tmp`) and deleted after the chain completes, ensuring no artefacts are left behind on success. On error, they may be retained for debugging.

3.4 Backward Compatibility

The manifest version is incremented. The upgrade function `upgrade_manifest_vN_to_vN+1` performs a data migration: for each `PreProcessor` defined in `custom_processors.preprocessors`, a corresponding one-step `PreProcessorChain` is created with the same name, the same `extension_filter`, and steps containing that preprocessor's name. Template references in `PreProcessors.preprocessors` are unchanged in value — they already hold the preprocessor name, which now matches the generated chain name. The result is transparent to existing projects: every prior single-preprocessor workflow becomes a one-step chain with identical runtime behaviour.

4. Complexity Assessment

Concern	Assessment
Model change	Low — one new struct, one new field on <code>Processors</code>
Execution logic	Medium — temp-file management, error-path cleanup
Parallelism	No change — chains run sequentially per chunk; chunks remain parallel
Manifest migration	Low — data transformation required, but fully automated
CLI surface	None — no new subcommands; chains are defined in the manifest
Breaking changes	None for existing projects

The most complex aspect is the temporary file lifecycle: files must survive long enough to be passed as CLI arguments and be cleaned up reliably even when intermediate steps fail. Rust's Drop-based RAII (e.g., `tempfile::NamedTempFile`) handles this cleanly.

5. Testability

5.1 Unit Tests

- `choose_chain` (replacement for `choose_preprocessor`) returns the correct `PreProcessorChain` by extension filter and fallback; tested for both cases.
- `run_preprocessor_chain` tested with mock steps using `cat` and `sed` — universally available on the target platform.
- Temp-file cleanup verified by asserting file absence after successful and failed runs.

5.2 Integration Tests

Integration tests follow the established pattern in `cli/tests/`. Relevant test scenarios:

1. **Two-step chain on .md files** — first step converts to an intermediate format, second step performs a substitution; output is compared to a known fixture.
2. **Chain with a failing intermediate step** — verifies that conversion returns an error and no partial output is written.
3. **Extension-filter interaction** — a chain is registered for `.md` and a one-step chain for `.tex`; both operate correctly in the same template.
4. **Single-element chain** — verifies behavioural equivalence with prior single-preprocessor behaviour.
5. **Manifest upgrade** — a vN manifest with single-preprocessor definitions is loaded; after upgrade, each preprocessor has a corresponding one-step chain and conversion produces identical output.

5.3 Existing Test Impact

Existing tests that exercise conversion paths go through `run_preprocessors_on_inputs`, which now always routes through `run_preprocessor_chain`. Since the upgrade migrates every prior preprocessor to an equivalent one-step chain, the runtime behaviour is identical and no test fixture changes are required. Tests that directly call `retrieve_preprocessors` or `choose_preprocessor` by name will require signature updates.

6. Use Cases

6.1 Format Normalisation Before Conversion

A project mixes Markdown files with slightly non-standard frontmatter. A first preprocessor step runs a small `sed`-based script to normalise the frontmatter syntax; the second step runs `pandoc`. Without chaining, the normalisation script must also invoke `pandoc` internally, coupling two concerns.

6.2 Post-Processing pandoc Output

In a `CustomPreprocessors` template, `pandoc` converts Markdown to HTML. A second step runs `html-tidy` or a custom minifier. The chain ensures the final output file is clean without requiring a wrapper script.

6.3 Multi-Stage Format Pipelines

A project stores content in AsciiDoc. A first step runs `asciidoctor` to produce DocBook XML; a second step runs `pandoc` to convert DocBook to the target format. Each tool is configured independently, and their combination is expressed declaratively in the manifest.

6.4 Metadata Injection

A first preprocessor step uses a small tool to inject computed metadata (e.g., a word count or a generated timestamp) into the file as YAML frontmatter. The second step is the standard `pandoc` conversion. This avoids the need for a Lua filter and keeps the injection logic in a reusable, language-agnostic tool.

6.5 PHP Template Rendering

A project uses `.php` source files as content templates — PHP scripts that emit Markdown when executed. A first preprocessor step invokes `php` to render the file to plain Markdown; a second step runs `pandoc` to convert that Markdown to the target format. This allows dynamic content (conditionals, loops, includes) to be resolved at conversion time without modifying the `pandoc` pipeline or embedding PHP awareness into the template system.

```
[[custom_processors.preprocessors]]
name      = "php_render"
cli       = "php"
cli_args  = []

[[custom_processors.preprocessors]]
name      = "pandoc_md_to_html"
cli_args  = ["-f", "markdown", "-t", "html"]

[[custom_processors.preprocessor_chains]]
name      = "php_to_html"
extension_filter = "php"
steps     = ["php_render", "pandoc_md_to_html"]
```

6.6 Linting as a Pipeline Step

A linting or validation tool is inserted as the first step in a chain. If it exits with a non-zero code, the chain fails and conversion halts with a clear error, without the need for a separate pre-conversion hook.

7. Limitations and Risks

- **Stdin vs. file-path semantics:** Some CLI tools do not read from files passed as positional arguments and require `stdin` or a named flag. The temp-file approach accommodates tools that accept a file path; tools that require `stdin` would need an additional `stdin_mode` flag on `PreProcessorStep` to route the intermediate content via `stdin` rather than a temp file.
- **Debugging opacity:** A failed intermediate step produces only the error from that step; the intermediate content at the point of failure may not be visible unless the temp file is retained. A `--keep-intermediates` debug flag would improve diagnostics.
- **Performance:** Each chain step is a separate process spawn. For short-running tools over many files this adds measurable overhead. Mitigation: chains are still parallelised at the chunk level.

8. Conclusion

Linear chaining of preprocessors is technically feasible with modest implementation effort. The primary changes are confined to `manifest_model.rs` (one new struct and field) and `converters/`

`common.rs` (chain execution logic and temp-file management). The feature introduces no breaking changes, integrates cleanly with the existing extension-filter dispatch, and substantially increases the expressiveness of the `CustomPreprocessors` conversion type. The use cases demonstrate clear, practical value for document-processing pipelines that require composing multiple independent tools.

Recommendation: Implement. The complexity is well-bounded, the test surface is straightforward, and the feature directly addresses a real limitation of the `CustomPreprocessors` conversion type.

User Generated Repository — Feasibility Study

Abstract

`TiefDownConverter` currently ships a fixed set of built-in preset templates and relies on manual file management for any user-defined content (templates, Lua filters, project structures). This study assesses the feasibility of a *User Generated Repository* (UGR): a hosted, community-curated registry from which users can pull reusable content into their projects and to which they can publish their own. The study covers repository architecture options, content classification, authentication and trust, manifest integration, CLI surface, testability, and practical use cases.

1. Background and Motivation

`TiefDownConverter`'s extensibility model is entirely local. A user who creates a sophisticated LaTeX template for novel typesetting, a Lua filter for cross-reference normalisation, or a complete project scaffold for academic papers has no mechanism to share that work with the broader `TiefDown` community beyond copying files manually. Conversely, a new user bootstrapping a project must either start from the sparse set of built-in presets or discover external files through informal channels.

A community repository addresses both gaps: it lowers the cost of sharing finished artifacts and lowers the cost of starting a new project by providing a curated, discoverable pool of reusable content.

2. Content Classification

The repository must accommodate several distinct artifact types, each with different structures and consumption semantics:

Artifact Type	Description	Consumed By
Template	A single <code>.tex</code> , <code>.typ</code> , or EPUB template directory	<code>tdc project templates add</code>
Lua Filter	One or more <code>.lua</code> files, optionally with a metadata header	<code>tdc project templates update --add-filters</code>
Project Scaffold	A full <code>manifest.toml</code> skeleton and optional directory structure	<code>tdc init</code>
Bundle	A named collection combining templates, filters, and scaffolding	Any of the above

Each artifact is identified by a *package name* (e.g., `lix-novel-a4`) and a *semantic version*. A package descriptor (a small TOML or JSON document) accompanies each artifact, declaring its name, version, type, author, description, license, and dependencies on external tools (e.g., `requires typst >= 0.11`).

3. Repository Architecture Options

3.1 Git-Backed Index (Recommended)

A dedicated public Git repository (e.g., `TiefDownConverter/tdc-registry`) holds:

- An index file (`registry.toml` or `registry.json`) mapping package names to versions and download URLs.
- One entry per package, pointing to a versioned archive hosted as a GitHub Release asset or an external URL supplied by the author.

The client fetches only the index on `tdc repo update`, resolves packages locally, and downloads individual archives on demand. This approach requires no custom backend infrastructure, is version-controlled by default, and leverages GitHub's CDN for distribution.

3.2 Dedicated REST API

A lightweight API server (e.g., written with Axum) exposes endpoints for search, metadata lookup, and archive download. Publishing requires authenticated POST requests. This provides richer query capabilities (full-text search, filtering by artifact type) but introduces infrastructure maintenance overhead and operational costs.

3.3 Recommendation

Begin with the Git-backed index. It requires no server infrastructure, is immediately auditable, and is sufficient for the expected scale of a focused document-tooling community. A migration path to a REST API exists if the index approach proves limiting; the CLI abstraction layer (see §5) would hide this transition from users.

4. Authentication and Trust

4.1 Publishing

Publishing is modelled on the crates.io contributor model: a contributor forks the registry repository, adds or updates their package entry, and opens a pull request. Maintainers review and merge. This makes every published package a conscious curatorial decision and avoids the need for a bespoke authentication server.

Automated publishing (e.g., from a CI pipeline) can be accommodated by a GitHub Actions workflow that validates the package descriptor, runs a schema check, and auto-merges if validation passes and the package version is new. The contributor's GitHub identity provides the authentication anchor.

4.2 Trust and Safety

Downloaded archives are verified against a SHA-256 checksum recorded in the index entry. On installation, `tdc` computes the checksum of the downloaded file and aborts if it does not match. This prevents corruption in transit and detects tampered hosted files.

Lua filters execute arbitrary code during pandoc conversion. The package descriptor must carry a `trust_required = true` flag for any artifact containing executable content. On first installation of such an artifact, the CLI prompts the user for explicit confirmation and displays the package's author, description, and source URL before proceeding.

5. CLI Surface

Five new subcommands are proposed, grouped under a repo top-level command:

```
tdc repo update           # Refresh the local index cache
tdc repo search <query>   # Search by name or description
tdc repo info <package>[@<version>] # Show package metadata
tdc repo pull <package>[@<version>] # Download and install into the current
project
tdc repo publish <path>    # Publish a local artifact to the registry
```

5.1 tdc repo update

Fetches the current `registry.toml` from the canonical registry URL (configurable via a global TiefDown settings file) and stores it in the local cache directory (`~/.config/tiefdownconverter/registry/`). Called automatically before search and info if the cache is stale (older than a configurable TTL, defaulting to 24 hours).

5.2 tdc repo pull

Resolves the requested package against the local cache, downloads the archive, verifies its checksum, and installs it according to its declared artifact type:

- **Template:** copies the template file(s) into `template/` and calls the existing `add-template` logic to register the entry in `manifest.toml`.
- **Lua Filter:** copies the filter file(s) into the project directory and optionally appends the filter to a specified template via `--template`.
- **Project Scaffold:** unpacks the scaffold into the project root; may optionally overwrite `manifest.toml` (requires `--force`).
- **Bundle:** processes each component in declaration order.

The installed package and its version are recorded in a `[repository]` section of `manifest.toml`, enabling `tdc repo update-packages` to check for newer versions in a future iteration.

5.3 tdc repo publish

Validates the local artifact against the package descriptor schema, computes the archive checksum, and guides the user through the submission process. In the Git-backed model, `publish` opens the browser to a pre-filled GitHub pull-request URL. In a future REST model, it would POST the archive and descriptor directly.

6. Manifest Integration

A new optional `[repository]` table records installed packages:

```
[repository]
registry_url = "https://raw.githubusercontent.com/TiefDownConverter/tdc-registry/main/registry.toml"

[[repository.installed]]
name      = "lix-novel-a4"
version   = "1.2.0"
type      = "template"

[[repository.installed]]
name      = "smart-quotes"
version   = "0.3.1"
type      = "lua_filter"
```

This table is optional and ignored by conversion commands; it exists solely as a record for tooling and future update-check functionality. No manifest version bump is required because the field is unknown to older versions and will be silently dropped on upgrade.

7. Complexity Assessment

Concern	Assessment
Registry infrastructure	Low (Git-backed); Medium (REST API)
CLI additions	Medium — five new subcommands, HTTP client, cache management
Archive handling	Low — zip/tar extraction; well-supported in Rust ecosystem
Checksum verification	Low — sha2 crate
Trust prompting	Low — one interactive confirmation gate
Manifest changes	Low — additive, optional table; no migration required
Cross-platform install paths	Medium — path handling differs between Windows and Unix
Testing	Medium — network calls require mocking or fixture servers

The most complex aspect is the install logic for bundles, which must compose multiple artifact-type-specific installation paths without leaving the project in a partially-installed state on failure.

8. Testability

8.1 Unit Tests

- Package descriptor deserialization: valid and invalid TOML/JSON fixtures.
- Index parsing: version resolution, checksum field presence.
- Checksum verification: known-good and tampered archives.
- Install path resolution per artifact type and platform.
- Cache staleness check: mock system time.

8.2 Integration Tests

The HTTP client is abstracted behind a `RegistryClient` trait with a real implementation and a test double. Integration tests use the test double, providing a fixture registry index and pre-built fixture archives:

Test	Scenario
<code>pull_template_registers_in_manifest</code>	pull installs a fixture template; <code>manifest.toml</code> lists it correctly
<code>pull_with_checksum_mismatch_aborts</code>	Tampered archive is rejected before any files are written
<code>pull_lua_filter_prompts_trust</code>	A <code>trust_required</code> filter prompts the user; denial aborts without installation
<code>pull_scaffold_populates_project</code>	A scaffold archive unpacks into the correct directory layout
<code>search_returns_matching_packages</code>	A fixture index with three entries; query matches one by name
<code>publish_validates_descriptor</code>	A malformed descriptor is rejected before submission

8.3 Existing Test Impact

No existing tests are affected. The repository commands are entirely additive; conversion, project management, and initialization paths are unchanged.

9. Use Cases

9.1 Novel Author Bootstrap

A user starting a new novel project runs:

```
tdc init --no-templates
tdc repo pull lix-novel-a4
tdc repo pull lix-novel-book
tdc repo pull chapter-heading-filter
```

Within three commands, the project is ready for conversion with professionally typeset chapter headings, without any manual template file management.

9.2 Academic Paper Template Sharing

A research group maintains a shared LaTeX template for their journal submissions. They publish it to the registry once. All group members run `tdc repo pull group-journal-template` to receive the same template, with version-locked reproducibility.

9.3 Lua Filter Ecosystem

Short, reusable Lua filters (smart-quotes normalisation, figure caption formatting, footnote rewriting) are published individually. Users assemble a per-project filter stack from these building blocks without duplicating filter code across repositories.

9.4 Project Scaffold for Technical Documentation

A software team maintains a scaffold that includes a `manifest.toml` with a documentation-specific template, a Lua filter for code block styling, and a shared metadata file with company-standard fields. New documentation projects are bootstrapped by:

```
tdc init --no-templates
tdc repo pull company-docs-scaffold --force
```

9.5 Typst Template Distribution

A designer publishes a visually polished Typst template. Because Typst templates do not require external LaTeX installations, the package can be adopted by any TiedDown user regardless of their LaTeX environment, widening the potential contributor and consumer base.

10. Limitations and Risks

- **Content quality:** The Git-backed pull-request model provides a curation gate but scales poorly if submission volume grows. A categories and quality tagging system would help discoverability without requiring deeper infrastructure.
 - **Registry centralisation:** A single canonical registry is a single point of failure. An optional `registry_url` override in the manifest and a global config file mitigates this; teams can self-host a private registry.
 - **Executable artifacts:** Lua filters execute arbitrary code. The trust-prompt gate is a usability friction point; users may habituate to confirming without reading. Clear, persistent provenance display (author, source URL, commit hash in the index) is the primary mitigation.
 - **Windows path handling:** Archive extraction and install-path construction must account for Windows path separators and reserved filenames. This is a known source of subtle bugs and requires explicit test coverage on Windows runners.
 - **Versioning strategy for project scaffolds:** Scaffolds do not naturally version the same way libraries do. Pulling a newer version of a scaffold into an existing project risks overwriting user edits. The safest default is to refuse overwriting and require `--force`.
-

11. Conclusion

A User Generated Repository is technically feasible with moderate implementation effort. The Git-backed index approach eliminates the need for custom server infrastructure while providing a fully auditable, version-controlled package history. The five proposed CLI subcommands integrate cleanly with the existing command structure. The primary engineering risks — cross-platform install paths and the trust surface of executable Lua artifacts — are well-understood and have established mitigations.

Recommendation: Implement, starting with the Git-backed index and pull/search/info commands. Publishing via pull-request can ship in the same iteration. An automated publishing API and update-check functionality are natural second-iteration additions once the registry has established adoption.

Website Generator Template — Feasibility Study

Abstract

TiedDownConverter is capable of acting as a static site generator (SSG): the TiedDownConverter documentation project itself is built with it, producing multi-page HTML output via CustomPreprocessors templates, Pandoc's HTML5 backend, injection-based headers and footers, and a Lua-driven navigation layer. That configuration is, however, the product of accumulated project-specific knowledge — it is not reusable, not documented as a starting point, and involves several non-obvious decisions (injection ordering, Lua navigation generation, multi-file output flags, metadata field conventions). This study evaluates whether shipping a first-class website preset — a self-contained, opinionated starting point for documentation sites — is warranted, what it would require, and where the genuine risks lie.

1. Background and Motivation

The TiedDownConverter README acknowledges that this tool is not aimed at users who want a GUI or a zero-configuration experience. Its power users are writers and developers comfortable with TOML configuration, Pandoc, and Lua. Within that audience, there is a real but narrow sub-population who want to publish documentation sites from Markdown sources: software projects, technical writing workflows, personal wikis.

Today, a user who wants to use TiedDownConverter as an SSG must:

1. Understand CustomPreprocessors vs. CustomProcessor template types.
2. Understand multi_file_output, output_extension, and combined_output semantics.
3. Understand the injection system (header, body, footer; ordering; that injections are inserted at the Pandoc input stage, not post-processed).
4. Write or adapt HTML injection files manually.
5. Author or copy a navigation Lua filter, and understand how meta_gen interacts with it.
6. Understand the metadata field pass-through mechanism (`--metadata title={{title}}`).

None of these steps are individually intractable, but the combination is high enough friction that a motivated user can spend a full day before producing anything that renders correctly. The existing docs project provides a working reference, but it is not structured as a reusable scaffold — it contains TiedDownConverter-specific navigation, content, and metadata.

A website preset would lower this barrier by providing a turnkey starting point: sensible defaults, working HTML/CSS, and a manifest skeleton a user can understand and extend without first understanding the full internals.

2. Proposed Scope

The preset is scoped to what can be shipped as static files inside the TiedDownConverter binary (alongside the existing LaTeX and Typst templates in `core/src/resources/templates/`) or as an init-time scaffold written to disk.

The minimal viable preset consists of:

Artifact	Description
manifest.toml skeleton	Pre-configured CustomPreprocessors template with correct multi-file and single-file output, a navigation filter, and a default profile
template/style.css	A CSS file providing readable, minimal styling
template/head.html	Injection: <code><head></code> open, <code><style></code> link or inline, <code><nav></code> open
template/footer.html	Injection: <code></body></html></code> close with a simple footer
template/nav.lua	Lua filter that inserts inter-page navigation links
Markdown/index.md	Seed content file so conversion runs immediately after init

A secondary goal, but not a requirement for the first iteration, is a `--site` flag on `tdc init` that invokes the preset and writes these files, as opposed to requiring the user to copy them from documentation.

3. Current Architecture Fit

3.1 What Already Works

The docs project demonstrates that the full SSG pipeline is functional today:

- CustomPreprocessors with `output_extension = "html"` and `multi_file_output = true` produces one HTML file per input Markdown file.
- CustomPreprocessors with `combined_output = "index.html"` collapses all inputs into a single page.
- Injections (`header_injections`, `footer_injections`) splice raw HTML around each converted chunk.
- `[templates.meta_gen] feature = "Full"` and `feature = "MetadataOnly"` drive the navigation Lua filter via generated metadata.
- `--metadata title={{title}}` passes project-level metadata into Pandoc per file.

None of these paths require modification to ship a preset.

3.2 What the Preset Must Work Around

Two genuine friction points exist that the preset cannot fully hide:

Absolute CSS paths. The docs manifest uses `--css /TiefDownConverter/template/html_template/style.css` — an absolute path hardcoded to the docs project's deployment environment. A general preset cannot hardcode a path that works for every user. Options:

- Inline the CSS directly in `head.html` (eliminates the path problem entirely; increases injection file size; makes CSS harder to override).
- Use a relative path and document that users must serve from the project root or adjust the path.
- Accept this limitation and document it clearly.

Inlining is the least surprising choice for a preset aimed at first-time users.

Navigation generation. The existing `navlib.lua` and `nav.md` approach is coupled to TiefDownConverter's own site structure. A general-purpose nav filter needs to operate on whatever files the user provides, constructing links from metadata rather than a hardcoded list. This requires either a generic nav Lua filter (moderate Lua work, ~100 lines) or accepting that navigation in the preset is manual (a `nav.md` the user edits). The latter is simpler and more honest about what TiefDownConverter actually automates.

4. Usefulness Assessment

4.1 Who Would Use This

The realistic audience is narrow. It is not people who are already using Jekyll, Hugo, or MkDocs — those tools are better SSGs, with themes, plugin ecosystems, live reload, and deployment integrations. The realistic audience is:

- Existing TiefDownConverter users who already produce PDF output and want HTML as an additional format without switching tools.
- Software developers who write documentation in Markdown, already use TiefDownConverter for some output, and want a quick static site without learning a second tool.

This is a useful but small population. TiefDownConverter is not competing with dedicated SSGs on features, and the preset should not pretend otherwise.

4.2 Practical Scenarios

Software project documentation. A developer uses TiefDownConverter to produce a PDF reference manual. Adding an HTML profile with the website preset gives them a deployable GitHub Pages site with no new tool dependency. The PDF and HTML share the same Markdown source.

Personal notes or wiki. A writer who already uses TiefDownConverter for long-form writing wants to publish a subset of their notes as a simple website. The preset gives them a working starting point in under five minutes.

TiefDownConverter onboarding showcase. The preset doubles as a demonstration of the injection and multi-file output systems. Users who initialise a site project and then read the generated `manifest.toml` learn about these features from a working example.

4.3 Limitations That Reduce Usefulness

- No live reload or development server. The user must re-run `tdc convert` and refresh the browser manually. This is a significant ergonomic gap compared to any dedicated SSG.
- No asset pipeline. Images, JavaScript, or other non-Markdown assets must be managed manually and copied into the output directory.
- Navigation is manual unless a generic nav filter is implemented. Users must maintain a `nav.md` or equivalent.
- Deployment is entirely manual. No integration with GitHub Pages, Netlify, or similar services.

These are not bugs — they reflect TiefDownConverter’s scope — but they mean the preset solves the initial-configuration problem, not the ongoing-workflow problem. A user who outgrows TiefDownConverter as an SSG will switch to a dedicated tool. The preset should be honest about this in its generated `manifest.toml` comments or in documentation.

5. Implementation Complexity

Concern	Effort
CSS and HTML injection files	Low — static files, no new code
Manifest skeleton	Low — TOML written once, embedded as a resource
<code>tdc init --site</code> flag (or <code>--template website</code>)	Low-Medium — wires into existing init path
Generic nav Lua filter	Medium — ~100 lines of Lua, needs testing across file counts
Relative CSS path workaround (inlining)	Low — authoring choice, not a code change
Documentation	Low-Medium — a page in the existing docs project

The total implementation effort for the minimal preset (without a generic nav filter) is low. The nav filter adds moderate effort and testing surface. The `tdc init` integration is the most likely place for edge-case bugs (path handling, existing file conflicts).

6. Testability

6.1 What Can Be Tested Automatically

- `tdc init --site` (or equivalent) creates the expected files in a temp directory.
- The generated `manifest.toml` parses correctly and passes manifest validation.
- `tdc convert` on the generated project succeeds and produces at least one `.html` file.

- The generated HTML contains the expected `<nav>` element (structural check, not visual).

6.2 What Cannot Be Tested Automatically

Visual styling correctness requires a browser. The CSS will not be pixel-perfect by default across browsers. A test that checks for the presence of structural HTML elements is the practical limit of automated verification. Styling regressions can only be caught by manual inspection or a snapshot-based browser test (which is out of scope for this project's test infrastructure).

7. Risks

Scope creep. An SSG preset naturally invites feature requests: themes, syntax highlighting, search, RSS, sitemap generation. The preset must be explicitly documented as an opinionated minimal starting point, not as a competing SSG. If the feature request volume is high, the cost of maintaining parity with user expectations will exceed the benefit.

CSS maintenance burden. Shipping a CSS file means owning it. Any future design change requires updating the file and documenting the change. If the preset's styling becomes dated or is perceived as low quality, it reflects on TiedDownConverter's presentation. The existing docs CSS (`head.html`) is polished but non-trivial in size. Shipping it as-is is reasonable but means any changes to the docs styling diverge from the preset unless they are kept in sync.

False advertising. The preset may set expectations that TiedDownConverter cannot meet as a general SSG. Users who initialise a site project and then discover that navigation requires manual maintenance, or that there is no dev server, may be disappointed. The docs and `manifest.toml` comments must set expectations correctly.

8. Comparison to Alternatives

A user who wants a static site from Markdown has better-supported alternatives: MkDocs (Python, excellent documentation tooling), Hugo (Go, fast, large theme ecosystem), or Jekyll (Ruby, GitHub Pages native). These tools are specifically designed for this use case and will outperform a TiedDownConverter preset on nearly every SSG-specific dimension.

The preset's value proposition is narrow: it is useful precisely for users who are already in the TiedDownConverter ecosystem and want HTML as one additional output format, not as the primary product. It is not a reason to choose TiedDownConverter as an SSG in a greenfield project.

9. Conclusion

A website generator template preset is technically trivial to implement in its minimal form — the pipeline already works, the assets exist in the docs project, and the manifest skeleton is a one-time authoring task. The genuine questions are whether the narrowness of the target audience justifies the ongoing CSS and documentation maintenance cost, and whether shipping an SSG preset is honest about TiedDownConverter's capabilities.

Recommendation: Implement the minimal preset, with caveats. The implementation cost is low enough that it earns its place as an onboarding aid and a multi-format complement for existing users. It should be explicitly framed as a starting point for users who already use TiedDownConverter, not as a general SSG recommendation. The generic nav filter should be deferred to a second iteration — manual navigation via a `nav.md` file is good enough for the initial preset and avoids the ongoing Lua maintenance surface. Do not add a live-reload dev server, asset pipeline, or deployment integration: if those become requested, the correct answer is to point users at a dedicated SSG.

Resource Copying to Output — Feasibility Study

Abstract

TiefDownConverter copies declared resources from the markdown project directory into the intermediate build directory during conversion, but these resources are not propagated to the final output directory. Users building websites or other asset-dependent targets that load resources dynamically at runtime must either copy images, stylesheets, and navigation files manually after each build, or author a dedicated Preprocessor to handle this. This study evaluates the feasibility of adding a configuration option to copy declared resources — and optionally generated navigation metadata files — directly to the output directory as part of the standard conversion pipeline.

1. Background and Motivation

The existing `resources` field on a `MarkdownProject` (type `Option<Vec<PathBuf>>`) specifies files and directories that live inside the markdown source directory and are needed during conversion. The `copy_resources` function in `core/src/conversion.rs` copies them into the intermediate build directory alongside the compiled template output. After conversion completes, `convert_template` copies only the primary output file (or directory, for `multi_file_output`) to the project's output directory. Resources are left behind in the timestamped build directory and discarded when smart-clean removes old builds.

This creates a friction point for any workflow that produces output consumed outside TiefDownConverter's build directory — most obviously HTML websites, where the browser resolves image and stylesheet URLs relative to the HTML file's location. A user who builds a multi-page documentation site using the website preset (feasibility study 3) and embeds a local image in a markdown page will find that image missing from the output directory after every build.

The two current mitigations are:

1. **Manual post-build copy** — fragile, error-prone, and breaks incremental workflows.
 2. **Custom Preprocessor** — functional but disproportionately complex for the task of copying files; it requires understanding the preprocessor system and authoring a shell script or binary that serves as a no-op conversion step with a side-effect of copying files.
-

2. Scope of the Proposed Feature

Three distinct categories of artefact are candidates for output-directory propagation:

Category	Description	Currently handled?
Declared resources	Files/directories listed in <code>resources</code> on a <code>MarkdownProject</code>	Copied to build dir only
Generated nav metadata	<code>nav.yml</code> / <code>nav.json</code> produced by the <code>meta_gen</code> pipeline	Written to build dir only
Template assets	Files from the <code>template/</code> directory (CSS, HTML fragments)	Copied to build dir only

This study addresses category 1 and 2. Category 3 is a separate concern: template assets are shared infrastructure and their propagation has different semantics (they may conflict across markdown projects sharing an output directory).

3. Use Cases

3.1 Static Website with Local Images

A user maintains a documentation site in TiedDownConverter. Pages reference images stored in `Markdown/images/`. The generated HTML files are served by a static host (GitHub Pages, Nginx, a local development server). After each `tdc convert`, the output directory must contain both the HTML files and the `images/` directory at the same relative path. Today this requires a post-build script.

This use case is real, recurring, and has no clean solution within the current feature set. It directly affects users following the website preset workflow introduced in feasibility study 3.

3.2 JavaScript or CSS Loaded Dynamically

A custom HTML template references a `prism.js` file and a `custom.css` stored alongside the markdown sources for version-control convenience. The conversion pipeline produces HTML that `<link>s` or `<script src>s` these files by relative path. Resource propagation would ensure they arrive in the output directory without user intervention.

This use case exists but is narrower than 3.1: users who are sophisticated enough to embed custom JS/CSS in their templates are typically also capable of writing a post-build copy step. It is a real convenience gain, not a blocker.

3.3 Navigation Metadata for Client-Side Rendering

A website uses a lightweight JavaScript navigation renderer that reads `nav.json` at runtime to build a sidebar. TiedDownConverter's `meta_gen` pipeline already produces this file in the build directory. Propagating it to the output directory would close the loop for this pattern.

This use case is plausible but currently limited to users who have already built around TiedDownConverter's `meta_gen` feature, which is itself relatively new. It is worth supporting but not the primary motivation.

3.4 Non-Web Outputs

For LaTeX/PDF or EPUB outputs, resources are embedded into the final document by the conversion process. There is no meaningful output-directory propagation to perform — the output is a single self-contained file. The feature would silently apply to these output types too, doing unnecessary file copies. This is harmless but worth noting: copying images to the same directory as a PDF output is useless and could surprise users.

4. Design Options

4.1 Option A: Boolean flag `copy_resources_to_output` on `MarkdownProject`

Add `copy_resources_to_output: Option<bool>` to the `MarkdownProject` struct. When true, after `convert_template` completes, the same resources already copied to the build directory are also copied to the project's output directory.

Pros: Minimal manifest surface area; opt-in; no breaking changes; isolated to the existing `copy_resources` call site.

Cons: Binary — no way to specify a subset of resources for propagation. Also does not address `nav` metadata files (§2 category 2) without an additional flag.

4.2 Option B: Separate output_resources list on MarkdownProject

Add `output_resources: Option<Vec<PathBuf>>` as a distinct list. Resources in `resources` are copied to the build directory only; resources in `output_resources` are copied to both.

Pros: Precise control; avoids propagating resources that are only needed during conversion (e.g., a `.bib` file for BibTeX). Cleanly extensible to cover nav metadata by including the generated file path.

Cons: Two lists to maintain; potentially confusing relationship between `resources` and `output_resources` for new users. Adding a resource to the wrong list is a silent failure.

4.3 Option C: Boolean flag plus copy_nav_to_output flag

Two separate boolean flags: `copy_resources_to_output` for declared resources, `copy_nav_to_output` for the generated nav file.

Pros: Granular, explicit, independently controllable.

Cons: Flag proliferation; still does not allow per-resource opt-in.

4.4 Recommendation

Option A is the most pragmatic starting point. The common case — “I want my images next to my HTML output” — is addressed by a single boolean. Nav metadata propagation can be handled by adding the nav file path to the `resources` list (since `meta_gen` writes it to a predictable location) or addressed in a follow-on iteration with a targeted `copy_nav_to_output` flag. The separation between build-only and output-propagated resources (Option B’s advantage) is a real concern, but in practice TiedDownConverter projects with non-web outputs already handle resources differently; the use case for fine-grained resource routing is niche enough not to justify the added manifest complexity at this stage.

5. Implementation Complexity

The core implementation is straightforward:

1. Add `copy_resources_to_output: Option<bool>` to `MarkdownProject` in `core/src/manifest_model.rs`.
2. In the `convert` loop in `core/src/conversion.rs`, after `convert_template` succeeds, call a new `copy_resources_to_output_dir` function that mirrors the existing `copy_resources` logic but targets `project.join(markdown_project.output)` instead of the build directory.
3. No manifest version bump is required because the new field is optional and additive; older versions will silently ignore it on round-trip.

The destination path requires care: the output directory must be created before copying, which `convert_template` already ensures. The copy itself is a near-verbatim reuse of the existing `copy_resources` implementation.

Nav metadata file propagation is slightly more involved: the file is generated inside the build directory by the `meta_gen` pipeline and its path is not available until after `convert_template` returns. The path is deterministic (`nav_output` from `MetaGenerationSettings`, defaulting to a fixed filename), so it can be reconstructed and copied without modifying the converter return type.

Concern	Effort
Manifest model change	Trivial
convert loop addition	Low
Path construction and safety checks	Low
Nav metadata propagation	Low–Medium
Manifest upgrade (not required)	None
Testing	Low–Medium

6. Testability

6.1 Unit Tests

The feature has no meaningful unit-testable units beyond what already exists — the logic is a conditional file copy with a path. Unit tests for path construction are of marginal value.

6.2 Integration Tests

Integration tests are the appropriate vehicle:

Test	Scenario
<code>copy_resources_to_output_copies_image</code>	Resource declared; flag true; image appears in output dir after conversion
<code>copy_resources_to_output_false_does_not_copy</code>	Flag false (default); resource absent from output dir
<code>copy_resources_to_output_directory_resource</code>	Resource is a directory; entire subtree appears in output dir
<code>copy_nav_to_output_propagates_nav_file</code>	meta_gen configured; nav file appears in output dir
<code>copy_resources_to_output_pdf_output_noop</code>	LaTeX template; resource copied but PDF output is unaffected

Existing conversion integration tests are unaffected — the flag defaults to `false` and the new code path is not entered.

The test infrastructure already supports temporary project directories with controllable manifests; new tests follow the established pattern in `cli/tests/convert_integration_test.rs`.

7. Risks and Limitations

7.1 Meaninglessness for Non-Web Outputs

Copying images next to a `.pdf` is useless. The flag is opt-in, so users who enable it for a project that also produces PDF output will get harmless but unnecessary file copies. Documentation should clearly state the intended use case.

7.2 Output Directory Pollution

If a user's output directory is the project root (i.e., `output = "."`), resource propagation writes files directly into the project root. This may interfere with version control or project structure expectations. This is not new — the existing output-file copy already has this behaviour — but adding resource directories amplifies the surface area.

7.3 Overwriting User-Modified Files

If a user manually edits a resource in the output directory (e.g., a CSS file they have locally patched), the next `tdc convert` will overwrite it with the source version. This is consistent with how `TiefDownConverter` treats all output but may surprise users who treat the output directory as a working directory.

7.4 Nav Metadata Path Coupling

Propagating the nav file requires knowing its output path, which is determined by `MetaGenerationSettings.nav_output`. If this field is absent, `TiefDownConverter` must apply the same defaulting logic used during generation. A mismatch between the generation default and the propagation default would silently result in the wrong file being copied or no file being found. This is a low-probability but non-trivial bug surface that requires careful implementation.

8. Conclusion

Resource copying to the output directory is a genuine usability gap with a direct, recurring impact on the primary non-document use case: HTML website generation. The implementation effort is low, the design is additive and non-breaking, and the feature composes cleanly with the existing resources and `meta_gen` systems.

The use case is real but narrow. The majority of `TiefDownConverter` users produce PDF or EPUB output where this feature is irrelevant. For the website use case — which feasibility study 3 actively encourages — the friction is significant enough to warrant addressing.

Recommendation: Implement, scoped to Option A (single boolean flag on `MarkdownProject`).

Nav metadata propagation should be included in the same iteration, handled as a second optional boolean (`copy_nav_to_output`) on `MetaGenerationSettings` rather than as an extension of the resources mechanism, to avoid coupling path-resolution logic across two subsystems. Defer per-resource granularity (Option B) unless a concrete use case emerges.

Resource Copying to Output — Implementation Plan

Overview

Add opt-in resource propagation to the conversion pipeline: a `copy_resources_to_output` boolean on `MarkdownProject` that, when enabled, copies declared resources to the output directory after conversion completes; and a `copy_nav_to_output` boolean on `MetaGenerationSettings` that does the same for the generated nav metadata file. Both flags default to false, preserving all existing behaviour.

1. Deliverables

Change	Location
copy_resources_to_output field on MarkdownProject	core/src/manifest_model.rs
copy_nav_to_output field on MetaGenerationSettings	core/src/manifest_model.rs
copy_resources_to_output_dir function	core/src/conversion.rs
Nav file propagation in convert_template caller	core/src/conversion.rs
Integration tests	cli/tests/convert_integration_test.rs (or a new file)

No CLI surface changes are required — the flags are manifest-only. No manifest version bump is required — both fields are optional and additive.

2. Manifest Model Changes

2.1 MarkdownProject

In core/src/manifest_model.rs, add one field to the MarkdownProject struct:

```
pub copy_resources_to_output: Option<bool>,
```

The field is placed after the existing resources field. Deserialization defaults to None, treated as false everywhere it is read.

2.2 MetaGenerationSettings

Add one field to MetaGenerationSettings:

```
pub copy_nav_to_output: Option<bool>,
```

Placed after the existing nav_output field.

No default-impl or migration function changes are required because both fields are Option and TOML deserialization leaves absent keys as None.

3. Conversion Logic

3.1 New function: copy_resources_to_output_dir

In core/src/conversion.rs, add:

```
fn copy_resources_to_output_dir(
    markdown_project: &MarkdownProject,
    output_dir: &PathBuf,
    markdown_dir: &PathBuf,
) -> Result<()> {
    for resource in markdown_project.resources.clone().unwrap_or_default() {
        let source = markdown_dir.join(&resource);

        if !source.exists() {
            return Err(eyre!(
                "Resource file {} does not exist.",
                source.display()
            ));
        }
    }
}
```

```

    }

    if source.is_dir() {
        debug!("Copying resource directory to output: {}", source.display());
        dir::copy(
            &source,
            output_dir,
            &dir::CopyOptions::new().overwrite(true).content_only(false),
        )?;
    } else {
        debug!("Copying resource file to output: {}", source.display());
        file::copy(
            &source,
            &output_dir.join(source.file_name().unwrap_or(std::ffi::OsStr::new("."))),
            &file::CopyOptions::new().overwrite(true),
        )?;
    }
}
Ok(())
}

```

This mirrors `copy_resources` exactly, targeting `output_dir` instead of the build directory. The duplication is intentional — the two call sites have different destinations and the shared logic is too thin to abstract usefully.

3.2 Call site in the convert loop

In the `convert` function, after the `convert_template(...)` call succeeds, add:

```

let output_dir = project.join(markdown_project.output.clone());

if markdown_project.copy_resources_to_output.unwrap_or(false) {
    copy_resources_to_output_dir(&markdown_project, &output_dir, &input_dir)?;
}

```

`input_dir` is already in scope (bound earlier in the loop as `project.join(markdown_project.path.clone())`).

3.3 Nav metadata propagation

After the resources copy block, add:

```

if let Some(meta_gen) = &template_original.meta_gen {
    if meta_gen.copy_nav_to_output.unwrap_or(false) {
        let nav_filename = meta_gen
            .nav_output
            .clone()
            .unwrap_or_else(|| PathBuf::from("nav.yml"));
        let nav_source =
            markdown_project.compiled_directory_path.join(&nav_filename);
        if nav_source.exists() {
            file::copy(
                &nav_source,
                &output_dir.join(nav_filename.file_name().unwrap_or_default()),
                &file::CopyOptions::new().overwrite(true),
            )?;
            debug!("Copied nav metadata file to output directory.");
        } else {
            warn!(

```

```

        "copy_nav_to_output is set but nav file '{}'" was not found in the
build directory.",
        nav_source.display()
    );
}
}
}

```

Note: `template_original` refers to the resolved `Template` struct (returned by `get_template_mapping_from_name`) which must be kept in scope past the `convert_template` call. Currently it is consumed or goes out of scope; rebind it before the call if necessary.

A missing nav file produces a warning rather than an error: `meta_gen` is not guaranteed to produce a nav file for every template type, and a missing file should not abort an otherwise successful conversion.

4. Integration Tests

Add tests covering the following scenarios. These follow the pattern established in `cli/tests/convert_integration_test.rs`.

	Test name	Setup	Assertion
copy_resources_to_output_copies_image_files		Project with one image resource; <code>copy_resources_to_output = true</code>	Image exists in output dir after conversion
copy_resources_to_output_default_false_does_not_copy		Same project; flag absent	Image absent from output dir
copy_resources_to_output_copies_directory		Resource is a subdirectory; flag <code>true</code>	Subdirectory and contents present in output dir
copy_nav_to_output_propagates_nav_files		<code>meta_gen</code> configured with <code>copy_nav_to_output = true</code>	<code>nav.yml</code> (or configured name) present in output dir
copy_nav_to_output_missing_file_warns		<code>copy_nav_to_output = true</code> but template type does not produce nav	Conversion succeeds; warning logged

5. Documentation

The two new fields should be documented in the manifest reference section of the project docs (if such a section exists) alongside the existing `resources` field and `meta_gen` settings. Usage guidance should note that the feature is primarily intended for HTML/website output; for PDF/EPUB outputs, enabling it copies files that will not be used.

6. Out of Scope

- Per-resource opt-in (separate `output_resources` list) — deferred pending a concrete use case.
- Template asset propagation (files from `template/`) — separate concern with different semantics.
- Automatic detection of output type to suppress no-op copies for non-web templates — over-engineering for the current scale of the feature.

Planning Documents

Below the planned feature plans, with detailed descriptions for development.

Linear Chaining of Preprocessors — Implementation Plan

Overview

Extend the preprocessor pipeline to support ordered, linear chains of preprocessors. The output of each step feeds as input to the next, enabling multi-tool workflows without requiring wrapper scripts. Chains become the sole dispatch unit; existing single-preprocessor configurations are migrated automatically to one-step chains during manifest upgrade.

1. Manifest Model (`core/src/manifest_model.rs`)

1.1 Add PreProcessorChain

```
/// An ordered sequence of preprocessor steps executed in series.
///
/// The stdout of step N is written to a temporary file and passed as input to step
N+1.
#[derive(Deserialize, Serialize, Clone)]
pub struct PreProcessorChain {
    pub name: String,
    pub extension_filter: Option<String>,
    pub steps: Vec<String>,
}
```

1.2 Extend Processors

Add the new field to Processors:

```
pub struct Processors {
    pub preprocessors: Vec<PreProcessor>,
    pub preprocessor_chains: Vec<PreProcessorChain>,
    pub processors: Vec<Processor>,
}
```

Default the field to an empty vec via `#[serde(default)]`.

1.3 Manifest Version Bump

Increment `CURRENT_MANIFEST_VERSION` in `core/src/consts.rs`. Add `upgrade_manifest_vN_to_vN+1` that iterates over every entry in `custom_processors.preprocessors` and creates a corresponding `PreProcessorChain` with the same name, the same `extension_filter`, and `steps = [preprocessor_name]`. The chain name matches the preprocessor name, so existing `PreProcessors.preprocessors` references in templates resolve correctly without modification. Register the upgrade in `upgrade_manifest`.

2. Resolution Layer (`core/src/converters/common.rs`)

2.1 Update retrieve_preprocessors

Change the return type from `Vec<PreProcessor>` to `Vec<PreProcessorChain>`. The function resolves each name in `PreProcessors.preprocessors` against `custom_processors.preprocessor_chains`. Update all call sites accordingly.

2.2 Replace choose_preprocessor with choose_chain

```
fn choose_chain(chains: &[PreProcessorChain], extension: &str) ->
Result<&PreProcessorChain>
```

Selection logic is identical to the current `choose_preprocessor`: prefer the entry whose `extension_filter` glob-matches extension; fall back to the entry with no filter. No dispatch branching is needed — the return type is always `PreProcessorChain`.

3. Execution Layer (`core/src/converters/common.rs`)

3.1 `run_preprocessor_chain`

```
fn run_preprocessor_chain(
    template: &Template,
    compiled_directory_path: &Path,
    metadata_fields: &Table,
    metadata_file: &Option<PathBuf>,
    nav_meta_data: &Option<(NavMeta, PathBuf)>,
    steps: &[PreProcessor],
    files: &[PathBuf],
) -> Result<String>
```

Algorithm:

1. Run `run_preprocessor` with the original files for `steps[0]`; capture result string.
2. For each subsequent step:
 - a. Write the intermediate string to `compiled_directory_path/.chain_intermediate_{step_index}.tmp` using `tempfile::NamedTempFile` (kept on disk via `keep()`).
 - b. Call `run_preprocessor` with the temp file path as the sole entry in `files`.
 - c. Delete the temp file.
3. Return the final result string.

Error handling: if any step fails, propagate the error immediately. Temp files created before the failure are cleaned up in the error path.

3.2 `Update run_preprocessors_on_inputs`

Replace the `choose_preprocessor + run_preprocessor` call with `choose_chain + run_preprocessor_chain`. No dispatch branching is needed:

```
let chain = choose_chain(&chains, &chunk.1)?;
run_preprocessor_chain(template, compiled_directory_path, ..., chain, &chunk.0)
```

The parallel `par_iter()` over chunks is unchanged. `run_preprocessor` becomes a private helper called only from within `run_preprocessor_chain`.

4. Dependency

Add `tempfile` to `core/Cargo.toml` if not already present:

```
[dependencies]
tempfile = "3"
```

Alternatively, construct temp file paths manually using a deterministic name derived from the thread ID or chunk index, and delete them explicitly. The `tempfile` crate is preferable for correctness on panic paths.

5. Test Plan

5.1 Unit Tests (core/src/_tests/)

File: core/src/_tests/preprocessor_chain_tests.rs

Test	Assertion
retrieve_chains_filters_by_name	Only chains whose names appear in <code>PreProcessors.preprocessors</code> are returned
retrieve_chains_unknown_name	Returns an error when a referenced name has no matching chain
choose_chain_by_extension	Extension-filtered chain is preferred over no-filter chain
choose_chain_fallback	No-filter chain selected when no filter matches the extension
choose_chain_no_match	Returns an error when no chain matches
run_chain_two_steps	cat followed by sed 's/a/b/' on a fixture file produces the expected output
run_chain_intermediate_cleanup	After a successful chain, no <code>.chain_intermediate_*.tmp</code> files remain in the compiled dir
run_chain_step_failure	A failing intermediate step causes <code>run_preprocessor_chain</code> to return <code>Err</code>
run_chain_single_step	A one-step chain produces the same output as a direct <code>run_preprocessor</code> call

5.2 Integration Tests (cli/tests/)

File: cli/tests/convert_preprocessor_chain_integration_test.rs

Test	Scenario
chain_two_step_md_conversion	Template with a two-step chain; fixture Markdown processed through cat + sed; output matches expected fixture
chain_failing_intermediate	Second step exits non-zero; <code>tdc convert</code> returns an error exit code
chain_extension_filter_coexistence	Template uses a two-step chain for <code>.md</code> and a one-step chain for <code>.tex</code> ; both produce correct output
chain_single_element	One-step chain produces identical output to the pre-migration single preprocessor behaviour
manifest_upgrade_migrates_preprocessors	Old manifest with single-preprocessor definitions loaded; after upgrade, each preprocessor has a matching one-step chain; conversion output is unchanged

6. Documentation

Update planning/Markdown/ with usage examples showing the new TOML syntax. Update any existing documentation that describes the `custom_processors` table to mention `preprocessor_chains`.

7. Delivery Sequence

1. Manifest model: add `PreProcessorChain`, extend `Processors`, bump version, add upgrade function with data migration.
2. Update `retrieve_preprocessors` return type; update all call sites to compile.
3. Replace `choose_preprocessor` with `choose_chain`.
4. Add `run_preprocessor_chain`; update `run_preprocessors_on_inputs` to call it exclusively.
5. Unit tests (red → green).
6. Integration tests (red → green).
7. Documentation update.

User Generated Repository — Implementation Plan

Overview

Add a community-curated package registry to `TiefDownConverter`, backed by a public Git repository index. Users can search, pull, and publish reusable templates, Lua filters, and project scaffolds. Five new `tdc repo` subcommands expose the registry surface. No existing conversion or project-management paths are modified.

1. Registry Repository (`TiefDownConverter/tdc-registry`)

Create a separate public GitHub repository. It contains:

```
tdc-registry/  
├─ registry.toml          # package index  
└─ packages/  
    └─ <name>/  
        └─ <version>.toml # per-version descriptor (optional, for auditing)
```

1.1 registry.toml Schema

```
[[packages]]  
name       = "lix-novel-a4"  
description = "A4 novel template using the LiX typesetting system."  
author     = "tiefseetauchner"  
license    = "MIT"  
type       = "template"          # template | lua_filter | scaffold | bundle  
trust_required = false  
  
[[packages.versions]]  
version = "1.2.0"  
url      = "https://github.com/example/releases/download/v1.2.0/lix-novel-a4.zip"  
checksum = "sha256:abcdef1234..."  
requires = []                  # e.g. ["typst>=0.11", "xelatex"]
```

The index is fetched as a single file; no recursive directory traversal is required.

2. Crate Structure Changes

2.1 New crate: `tdc-registry` (or module within `core`)

Given the existing separation between `core` and `cli`, registry logic belongs in `core` as a new module `core/src/registry/`:

```
core/src/registry/
├─ mod.rs
├─ client.rs      # RegistryClient trait + HTTP implementation
├─ index.rs       # registry.toml parsing and version resolution
├─ install.rs     # artifact installation per type
├─ cache.rs       # local index cache management
└─ publish.rs     # descriptor validation and submission preparation
```

A `RegistryClient` trait with a single real implementation (`HttpRegistryClient`) and a test double (`MockRegistryClient`) isolates network calls from all tests.

2.2 `cli/src/repo_commands.rs`

New file handling the five repo subcommand dispatch arms, mirroring the pattern in `cli/src/project_commands.rs`.

3. CLI Commands (`cli/src/cli.rs`)

Add a `Repo` variant to the top-level `Commands` enum:

```
#[command(about = "Interact with the TiefDownConverter community repository.")]
Repo {
    #[arg(help = "The project to apply repository operations to. Defaults to the
current directory.")]
    project: Option<PathBuf>,
    #[command(subcommand)]
    command: RepoCommands,
},
```

Add a new `RepoCommands` enum:

```
#[derive(Subcommand)]
pub(crate) enum RepoCommands {
    #[command(about = "Refresh the local package index cache.")]
    Update,

    #[command(about = "Search the registry for packages.")]
    Search {
        #[arg(help = "Search query (matched against name and description).")]
        query: String,
        #[arg(long, help = "Filter by artifact type (template, lua_filter, scaffold,
bundle).")]
        r#type: Option<String>,
    },

    #[command(about = "Show metadata for a package.")]
    Info {
        #[arg(help = "Package name, optionally with @version (e.g. lix-novel-
a4@1.2.0).")]
        package: String,
    },
},
```

```

#[command(about = "Download and install a package into the current project.")]
Pull {
    #[arg(help = "Package name, optionally with @version.")]
    package: String,
    #[arg(long, help = "For template packages, register under this template name
in the manifest.")]
    template_name: Option<String>,
    #[arg(long, help = "For lua_filter packages, add the filter to this
template.")]
    template: Option<String>,
    #[arg(long, help = "Overwrite existing files (required for scaffold packages
on existing projects).")]
    force: bool,
},

#[command(about = "Prepare and submit a package to the registry.")]
Publish {
    #[arg(help = "Path to the artifact or a package descriptor file.")]
    path: PathBuf,
    #[arg(long, help = "Package name (overrides descriptor field).")]
    name: Option<String>,
    #[arg(long, help = "Version (overrides descriptor field).")]
    version: Option<String>,
},
}

```

4. Registry Module Implementation

4.1 cache.rs

- Cache directory: ~/.config/tiefdownconverter/registry/ (resolved via the dirs crate).
- Cache file: index.toml.
- Staleness check: compare mtime of the cache file against a configurable TTL (default 86400 s). Exposed as is_stale() -> bool.
- load_or_fetch(client: &dyn RegistryClient) -> Result<Index>: returns the cached index if fresh, otherwise fetches and writes to disk.

4.2 index.rs

```

pub struct Index {
    pub packages: Vec<PackageEntry>,
}

impl Index {
    pub fn from_toml(src: &str) -> Result<Self>;
    pub fn search(&self, query: &str, artifact_type: Option<&str>) ->
Vec<&PackageEntry>;
    pub fn resolve(&self, name: &str, version: Option<&str>) ->
Result<&VersionEntry>;
}

```

Version resolution: if version is None, select the highest semver version. Use the semver crate for comparison.

4.3 client.rs

```

pub trait RegistryClient: Send + Sync {
    fn fetch_index(&self, url: &str) -> Result<String>;
}

```

```
fn download_archive(&self, url: &str) -> Result<Vec<u8>>;
}
```

HttpRegistryClient uses the ureq crate (already a likely dependency; if not, add it — it is a blocking HTTP client with no async runtime requirement, fitting TiedDown’s synchronous architecture). MockRegistryClient stores fixture strings and bytes in a HashMap.

4.4 install.rs

```
pub fn install_package(
    entry: &VersionEntry,
    archive_bytes: &[u8],
    project_path: &Path,
    opts: &InstallOptions,
) -> Result<InstalledRecord>
```

InstallOptions carries force, template_name, add_to_template. The function:

1. Verifies SHA-256 checksum of archive_bytes against entry.checksum. Fails immediately on mismatch.
2. If entry.trust_required, prints author, description, and source URL, and prompts for confirmation via dialoguer::Confirm. Aborts without writing any files on denial.
3. Extracts the archive to a temp directory using the zip crate.
4. Delegates to the type-specific installer:
 - install_template: copies file(s) to <project>/template/, calls existing core logic to register in manifest.toml.
 - install_lua_filter: copies .lua files to the project directory; if --template provided, appends filter paths to that template’s filter list in manifest.toml.
 - install_scaffold: merges the scaffold’s manifest.toml into the project’s (or overwrites with --force), copies auxiliary files.
 - install_bundle: iterates the bundle’s component descriptors and calls the relevant sub-installer for each.
5. Appends an [[repository.installed]] entry to the project’s manifest.toml.
6. Cleans the temp directory.

On any error after step 2 but before step 6, previously written files are rolled back (tracked by a Vec<PathBuf> of written paths, cleaned via fs::remove_file in the error path).

4.5 publish.rs

publish is a submission helper, not an upload command (in the Git-backed model):

1. Validates the package descriptor TOML/JSON against the schema (serde deserialization with deny_unknown_fields).
2. Computes the archive checksum.
3. Prints the complete descriptor that would be added to registry.toml.
4. Opens the browser to a pre-filled GitHub new-file URL targeting the tdc-registry repository, with the descriptor pre-populated in the URL’s value parameter (GitHub supports this for files under ~2 KB).

5. Manifest Model (core/src/manifest_model.rs)

Add an optional repository table. No version bump is needed; the field uses #[serde(default)] and is silently ignored by older versions:

```
#[derive(Deserialize, Serialize, Default, Clone)]
pub struct RepositorySettings {
    #[serde(default)]
    pub registry_url: Option<String>,
    #[serde(default)]
    pub installed: Vec<InstalledRecord>,
}
```

```
#[derive(Deserialize, Serialize, Clone)]
pub struct InstalledRecord {
    pub name: String,
    pub version: String,
    pub r#type: String,
}
```

Manifest gains `#[serde(default)] pub repository: RepositorySettings`.

6. Global Configuration

A global configuration file at `~/.config/tiefdownconverter/config.toml` stores the canonical registry URL and TTL:

```
registry_url = "https://raw.githubusercontent.com/TiefDownConverter/tdc-registry/
main/registry.toml"
cache_ttl_seconds = 86400
```

Resolved via a new `core/src/global_config.rs` module. If the file does not exist, built-in defaults are used.

7. Dependencies

Add to `core/Cargo.toml`:

```
ureq          = "2"           # blocking HTTP client
zip           = "2"           # archive extraction
sha2          = "0.10"        # checksum verification
semver        = "1"           # version resolution
dirs          = "5"           # platform-appropriate config/cache paths
dialoguer     = "0.11"        # interactive trust confirmation prompt
```

8. Test Plan

8.1 Unit Tests (core/src/_tests/registry_tests.rs)

Test	Assertion
index_parsing_valid_toml	A fixture registry.toml deserializes without error
index_search_by_name	Query matching one of three fixture packages returns exactly that package
index_search_by_type	type = "lua_filter" filter returns only matching entries
resolve_latest_version	No explicit version → highest semver version selected
resolve_exact_version	@1.0.0 resolves to the correct entry
resolve_unknown_package	Returns Err
checksum_mismatch_aborts	install_package with a tampered byte slice returns Err before writing any files
install_template_copies_files	Fixture zip with a single .tex file lands in template/; manifest.toml is updated
install_lua_filter_appends_to_template	Fixture zip with a .lua file appended to a named template's filters
install_scaffold_respects_force	Without --force, scaffold on an existing project returns Err; with --force, succeeds
install_rollback_on_failure	Simulated failure mid-install; no files remain in the project
cache_staleness_check	Mock mtime older than TTL → is_stale() returns true
trust_prompt_denial_aborts	Mock Confirm returning false → install_package returns Err without writing files

8.2 Integration Tests (cli/tests/repo_integration_tests.rs)

Test	Scenario
repo_pull_template_end_to_end	Fresh project; pull with mock client and fixture archive; template listed in manifest.toml
repo_pull_checksum_failure	Mock client returns archive with mismatched checksum; command exits non-zero
repo_search_prints_results	Fixture index with two packages; search output contains expected names
repo_info_prints_metadata	info lix-novel-a4@1.2.0 prints author, description, version, checksum
repo_pull_bundle_installs_all_components	Fixture bundle with one template and one filter; both are installed and registered
repo_update_writes_cache	update with mock client; cache file exists and contains fixture index after command

9. Documentation

Add a “Repository” section to docs/ covering:

- tdc repo command reference.
 - How to create and publish a package (descriptor schema, archive layout per artifact type).
 - How to self-host a private registry (pointing registry_url to a custom URL).
-

10. Delivery Sequence

1. Registry repository: create tdc-registry on GitHub with initial registry.toml (one or two seed packages).
2. core: add global_config.rs; add registry/ module skeleton with trait definitions and index.rs parsing.
3. core: implement cache.rs and client.rs; unit tests (red → green).
4. core: implement install.rs for the template artifact type only; unit tests (red → green).
5. cli: add RepoCommands to cli.rs; add repo_commands.rs with update, search, info, pull (template only) dispatch; integration tests for those four commands (red → green).
6. core: extend install.rs for lua_filter and scaffold artifact types; unit tests (red → green); extend integration tests.
7. core: implement publish.rs; add tdc repo publish CLI arm; unit test descriptor validation.
8. core: extend install.rs for bundle type; unit and integration tests (red → green).
9. Documentation update.

Website Generator Template — Implementation Plan

Overview

Add a website preset to tdc init that scaffolds a working static site project: a manifest.toml configured for multi-page HTML output, inlined CSS styling, HTML injections for a header and footer, a manual navigation file, and a seed content page. No new conversion pipeline code is required. The preset ships as static resource files embedded in the binary alongside the existing LaTeX and Typst templates.

1. Deliverables

File	Destination (relative to new project root)
Preset manifest skeleton	generated into manifest.toml at init time
head.html	template/head.html
footer.html	template/footer.html
nav.md	template/nav.md
index.md	Markdown/index.md

CSS is inlined into head.html to avoid absolute-path issues. No separate .css file is shipped.

2. Resource Files

2.1 head.html

A complete <!DOCTYPE html> ... <body><nav> fragment with inlined CSS. Base the styling on the existing docs/template/head.html (the glassmorphism card layout), stripping TiefDownConverter-

specific navigation links. The `nav` element closes immediately after its opening tag; the `nav` Lua filter or the `nav.md` injection fills it.

The file ends before `</nav>` so the `nav` injection content lands inside the `<nav>` element:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>{{title}}</title>
  <style>
    /* ... inlined CSS ... */
  </style>
</head>
<body>
  <nav>
```

2.2 `nav.md`

A Markdown file containing navigation links. The user edits this file to add links to their pages. It is included as a body injection (injected between `head.html` and the converted content, so it appears inside the `<nav>` element):

```
[Home](index.html) | [Page 2](page-2.html)
```

Users add new entries as they add new Markdown files. This is intentionally manual — auto-generated navigation is deferred.

2.3 `footer.html`

Closes the body and adds a minimal footer:

```
</article>
</main>
<footer>
  <p>Generated with <a href="https://github.com/Tiefseetauchner/
TiefDownConverter">TiefDownConverter</a></p>
</footer>
</body>
</html>
```

2.4 `Markdown/index.md`

A seed file so `tdc convert` succeeds immediately after `init`:

```
# My Site
```

Welcome to my site. Edit this file to get started.

3. Manifest Skeleton

The generated `manifest.toml` for a website project:

```
version = 6
smart_clean = true
smart_clean_threshold = 5

[[custom_processors.preprocessors]]
name      = "HTML"
cli_args  = [
```

```

    "--to", "html5",
    "--metadata", "title={{title}}",
    "--metadata", "author={{author}}",
]

[[injections]]
name = "site-header"
files = ["head.html", "nav.md"]

[[injections]]
name = "site-footer"
files = ["footer.html"]

[[markdown_projects]]
name = "Site"
path = "Markdown"
output = "site"

[shared_metadata]
title = "My Site"
author = ""

[[templates]]
name = "HTML Multi-Page"
template_type = "CustomPreprocessors"
multi_file_output = true
output = "site"
header_injections = ["site-header"]
footer_injections = ["site-footer"]

[templates.preprocessors]
output_extension = "html"
preprocessors = ["HTML"]

[[profiles]]
name = "default"
templates = ["HTML Multi-Page"]

```

The `nav.md` is in `header_injections` so it lands inside the `<nav>` element opened by `head.html`.

4. tdc init Integration

4.1 New Flag

Add `--site` to the `init` subcommand. When passed, `init` writes the website preset instead of the default no-template project.

```
tdc init --site [--output <dir>]
```

`--site` and `-t / --templates` are mutually exclusive; the CLI should error if both are provided.

4.2 Implementation Path

The existing `init` path in `cli/src/main.rs` (or wherever `init` is dispatched) calls into `core` to write a `manifest.toml` and copy template files. The `--site` branch:

1. Creates the project directory (same as today).

2. Writes `manifest.toml` from the website skeleton (either a hard-coded struct serialized to TOML, or a raw string constant).
3. Copies `head.html`, `footer.html`, `nav.md` from embedded resources into `template/`.
4. Writes Markdown/`index.md`.

The resource files are embedded via `include_str!` macros in `core/src/resources/` alongside the existing template files.

4.3 Existing Init Code Impact

The `--site` flag is additive. The existing init behavior is unchanged. No existing tests are affected.

5. Resource Embedding

Add to `core/src/resources/templates/` (or a new `core/src/resources/website/` directory):

```
core/src/resources/website/  
├─ head.html  
├─ footer.html  
├─ nav.md  
└─ index.md
```

Embed with `include_str!` constants in a new `core/src/website_preset.rs` module. This module exposes a single function:

```
pub fn write_website_preset(project_path: &Path) -> Result<()>
```

That function writes the four files to their destinations and returns the manifest struct for the caller to serialize.

6. Test Plan

6.1 Unit Tests

Test	Assertion
<code>website_preset_writes_all_files</code>	After calling <code>write_website_preset</code> , all four expected files exist at the correct paths
<code>website_manifest_pareses</code>	The website manifest constant deserializes without error using the existing manifest parser
<code>init_site_and_template_flags_are_exclusive</code>	Passing <code>--site -t template.tex</code> to the CLI exits non-zero with an appropriate error message

6.2 Integration Tests

Test	Assertion
<code>init_site_produces_convertible_project</code>	<code>tdc init --site</code> in a temp dir; <code>tdc convert</code> succeeds; <code>site/index.html</code> exists
<code>generated_html_contains_nav_element</code>	<code>site/index.html</code> contains a <code><nav></code> tag
<code>generated_html_contains_footer</code>	<code>site/index.html</code> contains a <code><footer></code> tag

The integration tests require Pandoc to be available on the test runner's PATH. These tests should be gated behind the same feature flag or environment variable used by other integration tests that invoke external tools.

7. Documentation

Add a “Getting started: documentation site” page to the TiedDownConverter docs project covering:

- When to use the website preset vs. a dedicated SSG (be direct: dedicated SSGs are better for complex sites).
- How to add pages (create a new `.md` file, add a link to `nav.md`).
- How to customize styling (edit the inlined CSS in `head.html`).
- How to deploy to GitHub Pages (brief — just point to GH Pages docs; TDC does not automate this).

8. Explicitly Out of Scope

The following are not part of this implementation and should be declined if raised as follow-up issues without a separate feasibility study:

- Live-reload development server.
- Automatic navigation generation from file structure.
- Asset pipeline (image copying, minification).
- Deployment integration (GitHub Actions, Netlify).
- Multiple themes or CSS customisation flags.

9. Delivery Sequence

1. Author `head.html`, `footer.html`, `nav.md`, `index.md` resource files. Convert locally with `tdc` to verify the HTML renders correctly in a browser.
2. Add `core/src/resources/website/` with embedded constants; add `core/src/website_preset.rs` with `write_website_preset`; unit tests (red → green).
3. Add `--site` flag to `init` subcommand in `cli`; wire to `write_website_preset`; integration tests (red → green).
4. Add `--site / -t` mutual-exclusion guard; test.
5. Add documentation page to the docs project.